

Actividad Guiada Video-Club

En esta actividad vamos a configurar el gestor de películas de un Video_Club con el que gestionar el estado de las diferentes películas y si están alquiladas o en cambio pertenecen a algún usuario.

1. Configuración del origen de datos

Lo primero que vamos a realizar es la configuración del origen de datos. Para ello haremos uso de EF Core que configurará la base de datos por nosotros con las diferentes tablas que vamos a necesitar.

Se van a usar 3 modelos para esta actividad:

- Película
- Cliente
- Categoría

Por tanto, tendremos que crear las clases que contendrán los atributos para cada una de ellas y de paso implementaremos la interfaz `INotifyPropertyChanged` para poder modificar datos en vivo a futuro en la interfaz.

Pelicula.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    public class Pelicula : INotifyPropertyChanged
    {
        // Atributos
        private int id;
        private string titulo = string.Empty;
        private string director = string.Empty;
        private int year;

        [Key]
        public int Id
        {
            get { return id; }
            set { id = value; OnPropertyChanged(); }
        }

        [Required, MaxLength(30)]
```

```

        public string Titulo
        {
            get { return titulo; }
            set { titulo = value; OnPropertyChanged(); }
        }

        [Required, MaxLength(30)]
        public string Director
        {
            get { return director; }
            set { director = value; OnPropertyChanged(); }
        }

        public int Year
        {
            get { return year; }
            set { year = value; OnPropertyChanged(); }
        }

        // Implementación de la interfaz INotifyPropertyChanged

        public event PropertyChangedEventHandler? PropertyChanged;

        protected void OnPropertyChanged([CallerMemberName] string? propertyName =
null)
        {
            PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
        }
    }
}

```

Para manejar a los clientes usaremos la clase `Cliente.cs`

`cliente.cs`

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    public class Cliente : INotifyPropertyChanged
    {
        // Atributos
        private int id;
    }
}

```

```
private string usuario = string.Empty;
private string nombreCompleto = string.Empty;
private string correoElectronico = string.Empty;
private int activo;
private DateTime fechaCreacion = DateTime.Now;
private DateTime fechaBaja = DateTime.MinValue;

[Key]
public int Id
{
    get { return id; }
    set { id = value; OnPropertyChanged(); }
}

[Required, MaxLength(30)]
public string Usuario
{
    get { return usuario; }
    set { usuario = value; OnPropertyChanged(); }
}

[MaxLength(100)]
public string NombreCompleto
{
    get { return nombreCompleto; }
    set { nombreCompleto = value; OnPropertyChanged(); }
}

[Required, MaxLength(100), EmailAddress]
public string CorreoElectronico
{
    get { return correoElectronico; }
    set { correoElectronico = value; OnPropertyChanged(); }
}

[Range(0, 1)]
public int Activo
{
    get { return activo; }
    set { activo = value; OnPropertyChanged(); }
}

public DateTime FechaCreacion
{
    get { return fechaCreacion; }
    set { fechaCreacion = value; OnPropertyChanged(); }
}

public DateTime FechaBaja
{
    get { return fechaBaja; }
    set { fechaBaja = value; OnPropertyChanged(); }
}
```

```

    }

    // Implementación de la interfaz INotifyPropertyChanged

    public event PropertyChangedEventHandler? PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string? propertyName =
null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
    }
}
}

```

Finalmente para poder asociar categorías a las películas, vamos a crear la clase categoría:

categoria.cs

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using System.Linq;
using System.Runtime.CompilerServices;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    public class Categoria : INotifyPropertyChanged
    {
        // Atributos
        private int id;
        private string clasificacion = string.Empty;

        [Key]
        public int Id
        {
            get { return id; }
            set { id = value; OnPropertyChanged(); }
        }

        [Required, MaxLength(30)]
        public string Clasificacion
        {
            get { return clasificacion; }
            set { clasificacion = value; OnPropertyChanged(); }
        }
    }
}

```

```
// Implementación de la interfaz INotifyPropertyChanged

public event PropertyChangedEventHandler? PropertyChanged;

protected void OnPropertyChanged([CallerMemberName] string? propertyName =
null)
{
    PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(propertyName));
}
}
}
```

Una vez creadas cada uno de los modelos vamos a configurar el `DbContext` para podernos conectar a la Base de Datos.

`VideoClubDbContext.cs`

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    public partial class VideoClubDbContext : DbContext
    {
        // MODELOS
        public virtual DbSet<Cliente> Clientes { get; set; }
        public virtual DbSet<Pelicula> Peliculas { get; set; }
        public virtual DbSet<Categoria> Categorias { get; set; }
        public VideoClubDbContext() { }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer("Data Source=(localdb)\\MSSQLLocalDB;Initial
Catalog=VideoClub");
        }
    }
}
```

1.1 Relación entre los modelos

Una vez tenemos las clases creadas y su conexión con la base de datos, tenemos que elaborar las relaciones entre las tablas, es aquí donde vamos a tener que modificar algunas líneas de estos documentos, difiriendo de los ejemplos vistos hasta ahora.

En las bases de datos relacionales, las relaciones definen mediante claves externas. Una clave externa (FK) es una columna o combinación de columnas que se utiliza para establecer e imponer un vínculo entre los datos de dos tablas.

Hay tres tipos de relaciones: uno a uno (1:1), uno a muchos (1:N) y muchos a muchos (N:M).

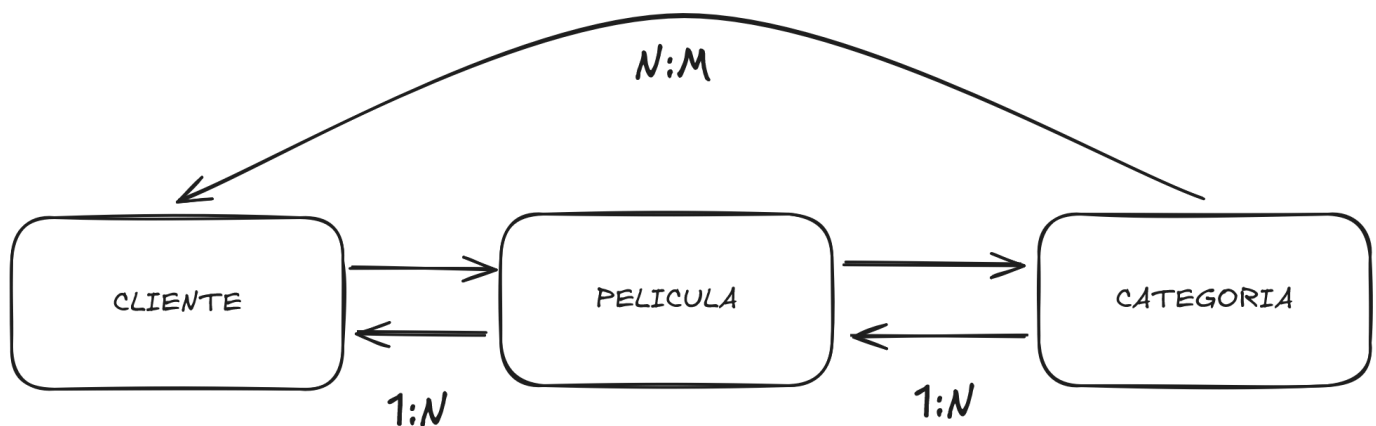
En una relación uno a muchos, la clave externa se define en la tabla que representa los varios extremos de la relación. La relación de muchos a muchos implica definir una tercera tabla (denominada unión o tabla de combinación), cuya clave principal se compone de las claves externas de ambas tablas relacionadas. En una relación uno a uno, la clave principal actúa además como una clave externa y no hay ninguna columna de clave externa independiente para ninguna tabla.

Para implementar las claves ajenas, *foreign key*, usaremos el atributo [ForeignKey].

Para la correcta implementación:

- Propiedad que contendrá el ID en la base de datos (int/no nullable para relación obligatoria, int? para opcional).
- Estableceremos la anotación [ForeignKey("Nombre_Modelo")] y la propiedad con el mismo tipo al referenciado.
- También debemos/podemos añadir una propiedad Navegacional, que consiste en una propiedad del tipo colección del modelo referenciado para facilitar la carga de datos.

Por este motivo vamos a configurar nuestras clases para que sigan el siguiente diagrama:



Por tanto, para conseguir este resultado vamos a crear las claves foráneas en la clase `Pelicula.cs` y configuraremos propiedades navegacionales en las otras dos.

Añadimos a `Pelicula.cs`

```
// Relaciones entre tablas

// FK hacia Cliente (1:N)
public int? ClienteId { get; set; } // Nullable, porque puede no estar alquilada
[ForeignKey("ClienteId")]
public virtual Cliente? Cliente { get; set; }

// FK hacia Categoría (1:N)
public int CategoriaId { get; set; }
```

```
[ForeignKey("CategoriaId")]
public virtual Categoria Categoria { get; set; } = null!;
```

Añadimos a `Cliente.cs`

```
// Relaciones entre tablas

// Relación 1:N con Películas
[InverseProperty("Cliente")]
public virtual ICollection<Pelicula> Peliculas { get; set; } = new List<Pelicula>();

// Relación N:M con Categorías
[InverseProperty("Clientes")]
public virtual ICollection<Categoria> Categorias { get; set; } = new List<Categoria>
();
```

Añadimos a `Categoria.cs`

```
// Relaciones entre tablas

// Relación 1:N con Películas
[InverseProperty("Categoria")]
public virtual ICollection<Pelicula> Peliculas { get; set; } = new List<Pelicula>();

// Relación N:M con Clientes
[InverseProperty("Categorias")]
public virtual ICollection<Cliente> Clientes { get; set; } = new List<Cliente>();
```

Además, para que EF Core entienda que hay que crear una cuarta tabla en la BBDD que relaciones clientes y categorías, debemos añadir lo siguiente al `DbContext` :

`VideoClubDbContext.cs`

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Configuración de relación muchos a muchos entre Cliente y Categoria
    modelBuilder.Entity<Cliente>()
        .HasMany(c => c.Categorias)
        .WithMany(cat => cat.Clientes)
        .UsingEntity(j => j.ToTable("ClienteCategoria"));

    base.OnModelCreating(modelBuilder);
}
```

1.2 Cargas de datos

Hecho esto, ya podemos realizar los `Service` de cada una de las clases, pero para ello hay que primero tener en cuenta el tipo de carga de datos que queremos realizar.

Hay de tres tipos:

Como hemos visto, se puede añadir una propiedad navegacional para mejorar la carga de datos en la tabla referenciada. Para realizar esta carga disponemos de dos aproximaciones que son la carga diferida (*lazy loading*), la carga diligente (*eager loading*) y la carga explícita (*explicit loading*).

1.2.1 Carga Diferida (**ESTE EJEMPLO NO SE COPIA EN EL PROYECTO**)

Significa que los datos relacionados se cargan de forma transparente desde la base de datos cuando se accede a la propiedad de navegación.

Se realiza de manera automática, para su implementación deberemos modificar el DbContext añadiendo la instrucción UseLazyLoadingProxies() al configurar el modelo.

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    => optionsBuilder
        .UseLazyLoadingProxies()
        .UseSqlServer(myConnectionString);
```

1.2.2 Carga diligente (**ESTE EJEMPLO NO SE COPIA EN EL PROYECTO**)

Significa que los datos relacionados se cargan desde la base de datos como parte de la consulta inicial.

Puede usar el método Include para especificar los datos relacionados que se incluirán en los resultados de la consulta.

A la hora de realizar la consulta debemos añadir el include

```
using (var context = new BloggingContext())
{
    var blogs = context.Blogs
        .Include(blog => blog.Posts)
        .ToList();
}
```

También podemos realizar múltiples niveles de anidación o varias relaciones.

```
using (var context = new BloggingContext())
{
    var blogs = context.Blogs
        .Include(blog => blog.Posts)
        .ThenInclude(post => post.Author)
        .Include(blog => blog.Owner)
        .ToList();
}
```

1.2.3 Carga explícita (**ESTE EJEMPLO NO SE COPIA EN EL PROYECTO**)

Significa que los datos relacionados se cargan explícitamente desde la base de datos en un momento posterior.

Puede cargar de manera explícita una propiedad de navegación a través de la API

`DbContext.Entry(...)`.

```
using (var context = new BloggingContext())
{
    var blog = await context.Blogs
        .SingleAsync(b => b.BlogId == 1);
    await context.Entry(blog)
        .Collection(b => b.Posts)
        .LoadAsync();
    await context.Entry(blog)
        .Reference(b => b.Owner)
        .LoadAsync();
}
```

1.3 Los Services

En nuestro caso y por temas de rendimiento vamos a emplear la carga diligente, ya que de esta manera solo cargaremos los datos que necesitemos, cuando los necesitemos.

Esta carga solo nos afecta a la hora de hacer las peticiones con LINQ y como hasta ahora hemos visto, esto solo ocurre en los Services que es donde configuramos el CRUD de nuestro modelos.

Vamos a ello:

`ServicePelicula.cs`

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    class ServicePelicula : IDisposable
    {
        bool disposed;
        public ServicePelicula()
        {
            disposed = false;
        }

        // METODOS CRUD

        // LISTAR TODAS LAS PELÍCULAS CON CLIENTE Y CATEGORÍA
        public async Task<List<Pelicula>> Listar()
        {
            using (var _context = new VideoClubDbContext())
            {

```

```

        return await _context.Peliculas
            .Include(p => p.Cliente) // Carga el cliente que alquila
la pelicula
            .Include(p => p.Categoria) // Carga la Categoria asicada a
la pelicula
            .AsNoTracking() // Propiedad de Linq para mejorar
el rendimiento cuando no se modifican los datos
            .OrderBy(p => p.Id) // Ordena los resultado por Id
            .ToListAsync(); // Ejecuta la consulta y devuelve
los resultados, es asincrono por lo que devuelve una Task<List<Pelicula>>
    }
}

// BUSCAR POR ID
public async Task<Pelicula?> Listar(int id)
{
    using (var _context = new VideoClubDbContext())
    {
        return await _context.Peliculas
            .Include(p => p.Cliente) // Carga el cliente que
alquila la pelicula
            .Include(p => p.Categoria) // Carga la Categoria
asicada a la pelicula
            .AsNoTracking()
            .FirstOrDefaultAsync(p => p.Id == id); // Devuelve el primer
objeto que cumpla la condicion de la expresion lambda
    }
}

// BUSCAR POR TÍTULO
public async Task<Pelicula?> Listar(string titulo)
{
    using (var _context = new VideoClubDbContext())
    {
        return await _context.Peliculas
            .Include(p => p.Cliente) // Carga el cliente que alquila
la pelicula
            .Include(p => p.Categoria) // Carga la Categoria asicada a
la pelicula
            .AsNoTracking()
            .FirstOrDefaultAsync(p => p.Titulo == titulo);
    }
}

public async Task<Pelicula> Insertar(Pelicula peli)
{
    if (peli == null) throw new ArgumentNullException(nameof(peli));

    using (var _context = new VideoClubDbContext())
    {
        await _context.Peliculas.AddAsync(peli);
        await _context.SaveChangesAsync();
        return peli;
    }
}

```

```

    }
}

public async Task<bool> Actualizar(Pelicula peli)
{
    if (peli == null) throw new ArgumentNullException(nameof(peli));

    using (var _context = new VideoClubDbContext())
    {
        var existing = await _context.Peliculas.FirstOrDefaultAsync(p => p.Id
== peli.Id);
        if (existing is null) return false;

        // Actualiza campos permitidos
        existing.Titulo = peli.Titulo;
        existing.Director = peli.Director;
        existing.Year = peli.Year;
        existing.ClienteId = peli.ClienteId;
        existing.CategoriaId = peli.CategoriaId;

        await _context.SaveChangesAsync(); // Llamamos directamente a
SaveChangesAsync gracias a que EF mantiene el objeto EXISTING trakeado con lo que se
llama Change Tracking
        return true;
    }
}

public async Task<bool> Borrar(int id)
{
    using (var _context = new VideoClubDbContext())
    {
        var entity = await _context.Peliculas.FirstOrDefaultAsync(p => p.Id
== id);
        if (entity is null) return false;

        _context.Peliculas.Remove(entity); // Marca el objeto entity para ser
eliminado en el siguiente SaveChangesAsync()
        await _context.SaveChangesAsync();
        return true;
    }
}

// MÉTODOS DE LIBERACIÓN DE RECURSOS -----

public void Dispose()
{
    Dispose(true);
    GC.SuppressFinalize(this);
}

protected virtual void Dispose(bool disposing)

```

```

    {
        if (disposed)
            return;

        if (disposing)
        {
        }

        disposed = true;
    }

    ~ServicePelicula()
    {
        Dispose(false);
    }
}

```

El siguiente service que vamos a configurar es el de clientes, aunque en este ejemplo no vamos a hacer uso de todo el CRUD

ServiceCliente.cs

```

using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    internal class ServiceCliente : IDisposable
    {
        // Campo para saber si ya se liberaron los recursos
        bool disposed;
        // Creamos un constructor vacio
        public ServiceCliente()
        {
            // Flag: Se ha llamado al método Dispose?
            disposed = false;
        }

        // METODOS CRUD

        public async Task<List<Cliente>> Listar()
        {
            using (var _context = new VideoClubDbContext())

```

```

    {
        return await _context.Clientes
            .AsNoTracking()
            .OrderBy(c => c.Id)
            .ToListAsync();
    }
}

public async Task<Cliente?> Listar(int id)
{
    using (var _context = new VideoClubDbContext())
    {
        return await _context.Clientes
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.Id == id);
    }
}

public async Task<Cliente?> Listar(string usuario)
{
    // Listamos un usuario por Nombre
    using (var _context = new VideoClubDbContext())
    {
        return await _context.Clientes
            .AsNoTracking()
            .FirstOrDefaultAsync(c => c.Usuario == usuario);
    }
}

public async Task<Cliente> Insertar(Cliente client)
{
    // Insertamos un nuevo usuario
    if (client == null) throw new ArgumentNullException(nameof(client));

    using (var _context = new VideoClubDbContext())
    {
        await _context.Clientes.AddAsync(client); // Agrega un nuevo objeto
        // User al DbSet y lo inserta de forma asincrona
        await _context.SaveChangesAsync(); // Ejecuta todas las operaciones
        // pendientes en la base de datos
        return client;
    }
}

public async Task<bool> Actualizar(Cliente client)
{
    // Actualizamos un usuario existente
    if (client == null) throw new ArgumentNullException(nameof(client));

    using (var _context = new VideoClubDbContext())
    {
        var existing = await _context.Clientes.FirstOrDefaultAsync(c => c.Id
        == client.Id);
    }
}

```

```

        if (existing is null) return false;

        // Actualiza campos permitidos
        existing.Usuario = client.Usuario;
        existing.NombreCompleto = client.NombreCompleto;
        existing.CorreoElectronico = client.CorreoElectronico;
        existing.Activo = client.Activo;
        existing.FechaCreacion = client.FechaCreacion;
        existing.FechaBaja = client.FechaBaja;

        await _context.SaveChangesAsync(); // Llamamos directamente a
SaveChangesAsync gracias a que EF mantiene el objeto EXISTING trakeado con lo que se
llama Change Tracking
        return true;
    }
}

public async Task<bool> Borrar(int id)
{
    using (var _context = new VideoClubDbContext())
    {
        var entity = await _context.Clientes.FirstOrDefaultAsync(c => c.Id ==
id);

        if (entity is null) return false;

        _context.Clientes.Remove(entity); // Marca el objeto entity para ser
eliminado en el siguiente SaveChangesAsync()
        await _context.SaveChangesAsync();
        return true;
    }
}

// MÉTODOS DE LIBERACIÓN DE RECURSOS -----

// Método público de IDisposable
public void Dispose()
{
    Dispose(true); // Método que libera los recursos cuando terminas de usar
la clase
    GC.SuppressFinalize(this); // Evita llamar al finalizador dos veces
}

// Método protegido: libera los recursos
protected virtual void Dispose(bool disposing)
{
    if (disposed)
        return;

    if (disposing)
    {
        //Liberar recursos no manejados como ficheros, conexiones a bd, etc.
    }
}

```

```

        disposed = true;
    }

    // Finalizador (por si se olvida llamar a Dispose), tambien conocido como
    Destructor
    ~ServiceCliente()
    {
        Dispose(false);
    }
}
}

```

Finalmente configuramos el Service para Categorías:

ServiceCategoria.cs

```

using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Video_Club
{
    internal class ServiceCategoria : IDisposable
    {
        // Campo para saber si ya se liberaron los recursos
        bool disposed;
        // Creamos un constructor vacio
        public ServiceCategoria()
        {
            // Flag: Se ha llamado al método Dispose?
            disposed = false;
        }

        // METODOS CRUD

        public async Task<List<Categoria>> Listar()
        {
            using (var _context = new VideoClubDbContext())
            {
                return await _context.Categorias
                    .AsNoTracking()
                    .OrderBy(cat => cat.Id)
                    .ToListAsync();
            }
        }

        public async Task<Categoria?> Listar(int id)
        {

```

```

        using (var _context = new VideoClubDbContext())
        {
            return await _context.Categorias
                .AsNoTracking()
                .FirstOrDefaultAsync(cat => cat.Id == id);
        }
    }

    public async Task<Categoria?> Listar(string clasificacion)
    {
        using (var _context = new VideoClubDbContext())
        {
            return await _context.Categorias
                .AsNoTracking()
                .FirstOrDefaultAsync(cat => cat.Clasificacion == clasificacion);
        }
    }

    // MÉTODOS DE LIBERACIÓN DE RECURSOS -----

    // Método público de IDisposable
    public void Dispose()
    {
        Dispose(true); // Método que libera los recursos cuando terminas de usar
        la clase
        GC.SuppressFinalize(this); // Evita llamar al finalizador dos veces
    }

    // Método protegido: libera los recursos
    protected virtual void Dispose(bool disposing)
    {
        if (disposed)
            return;

        if (disposing)
        {
            //Liberar recursos no manejados como ficheros, conexiones a bd, etc.
        }

        disposed = true;
    }

    // Finalizador (por si se olvida llamar a Dispose), tambien conocido como
    Destructor
    ~ServiceCategoria()
    {
        Dispose(false);
    }
}

```

Una vez tenemos los services configurados ya podemos aplicar los cambios en la base de datos y continuar con el desarrollo de nuestra interfaz:

```
Add-Migration InitialMigration
Update-Database
```

También, es aconsejable acompañar los proyectos de un script SQL para hacer una carga de datos de ejemplo, aquí os dejo uno de ejemplo:

datos.sql

```
-- Crear tabla Categorias si no existe
IF OBJECT_ID('dbo.Categorias', 'U') IS NULL
BEGIN
    CREATE TABLE Categorias (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        Clasificacion NVARCHAR(30) NOT NULL
    );
END
GO

IF OBJECT_ID('dbo.Clientes', 'U') IS NULL
BEGIN
    CREATE TABLE Clientes (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        Usuario NVARCHAR(30) NOT NULL,
        NombreCompleto NVARCHAR(100) NOT NULL,
        CorreoElectronico NVARCHAR(100) NOT NULL,
        Activo INT NOT NULL,
        FechaCreacion DATETIME2 NOT NULL,
        FechaBaja DATETIME2 NOT NULL
    );
END
GO

IF OBJECT_ID('dbo.ClienteCategoria', 'U') IS NULL
BEGIN
    CREATE TABLE ClienteCategoria (
        CategoriasId INT NOT NULL,
        ClientesId INT NOT NULL,
        CONSTRAINT PK_ClienteCategoria PRIMARY KEY (CategoriasId, ClientesId),
        CONSTRAINT FK_ClienteCategoria_Categorias FOREIGN KEY (CategoriasId)
REFERENCES Categorias(Id) ON DELETE CASCADE,
        CONSTRAINT FK_ClienteCategoria_Clientes FOREIGN KEY (ClientesId) REFERENCES
Clientes(Id) ON DELETE CASCADE
    );
END
GO
```

```

IF OBJECT_ID('dbo.Peliculas', 'U') IS NULL
BEGIN
    CREATE TABLE Peliculas (
        Id INT IDENTITY(1,1) PRIMARY KEY,
        Titulo NVARCHAR(30) NOT NULL,
        Director NVARCHAR(30) NOT NULL,
        Year INT NOT NULL,
        ClienteId INT NULL,
        CategoriaId INT NOT NULL,
        CONSTRAINT FK_Peliculas_Categorias_CategoriaId FOREIGN KEY (CategoriaId)
REFERENCES Categorias(Id) ON DELETE CASCADE,
        CONSTRAINT FK_Peliculas_Clientes_ClienteId FOREIGN KEY (ClienteId) REFERENCES
Clientes(Id)
    );
END
GO

```

```

-----
-- Inserciones de ejemplo
-----

```

```

INSERT INTO Clientes (Usuario, NombreCompleto, CorreoElectronico, Activo,
FechaCreacion, FechaBaja) VALUES
('Disponible', 'Disponible', 'disponible@videoclub.local', 1, '2025-01-01T00:00:00',
'0001-01-01T00:00:00'),
('laura.gomez', 'Laura Gómez', 'laura.gomez@example.com', 1, '2024-06-12T10:15:00',
'0001-01-01T00:00:00'),
('carlos.ruiz', 'Carlos Ruiz', 'carlos.ruiz@example.com', 1, '2023-11-01T09:00:00',
'0001-01-01T00:00:00'),
('maria.fernandez', 'María Fernández', 'maria.fernandez@example.com', 1, '2022-02-
20T14:30:00', '0001-01-01T00:00:00'),
('javier.santos', 'Javier Santos', 'javier.santos@example.com', 1, '2024-01-
05T12:00:00', '0001-01-01T00:00:00'),
('lucia.martin', 'Lucía Martín', 'lucia.martin@example.com', 1, '2021-08-
18T08:45:00', '0001-01-01T00:00:00');
GO

```

```

-----
-- Categorías (6)
-----

```

```

INSERT INTO Categorias (Clasificacion) VALUES
('Acción'),
('Comedia'),
('Drama'),
('Ciencia Ficción'),
('Terror'),
('Aventura');
GO

```

```

-----
-- Películas (30 registros)
-- Nota: ClienteId = 1 => "Disponible" (libre)
--       ClienteId > 1 => alquilada por ese Cliente

```

-- Categorical entre 1 y 6 según las inserciones anteriores

```
INSERT INTO Peliculas (Titulo, Director, Year, ClienteId, CategoriaId) VALUES
('El Último Héroe', 'John McTiernan', 1999, 2, 1),
('Risas a Medianoche', 'Todd Phillips', 2018, 1, 2),
('Corazones Rotos', 'Greta Gerwig', 2020, 3, 3),
('Misión Galáctica', 'James Cameron', 2015, 1, 4),
('El Grito Final', 'Jordan Peele', 2019, 4, 5),
('Aventuras en la Isla', 'Steven Spielberg', 2001, 1, 6),
('El Último Aliento', 'Ridley Scott', 2017, 5, 4),
('Humor sin Fronteras', 'Taika Waititi', 2021, 1, 2),
('La Sombra del Pasado', 'Christopher Nolan', 2006, 6, 3),
('Horizonte de Acero', 'Neill Blomkamp', 2010, 1, 4),
('La Noche del Lobo', 'Ari Aster', 2022, 2, 5),
('Exploradores del Futuro', 'Denis Villeneuve', 2019, 1, 4),
('Golpe Perfecto', 'Guy Ritchie', 2023, 3, 1),
('Cita a Ciegas', 'Nancy Meyers', 2003, 1, 2),
('Sueños Rotos', 'Damien Chazelle', 2016, 1, 3),
('El Abismo', 'James Cameron', 1989, 4, 4),
('Risas Mortales', 'Jordan Peele', 2021, 1, 2),
('Día de Furia', 'Tony Scott', 1998, 5, 1),
('Caminos Cruzados', 'Richard Linklater', 2014, 1, 3),
('Más Allá del Miedo', 'David Fincher', 2020, 6, 5),
('El Reino Perdido', 'Peter Jackson', 2002, 1, 6),
('La Máquina del Tiempo', 'Robert Zemeckis', 1990, 1, 4),
('Comedia Urbana', 'Edgar Wright', 2017, 2, 2),
('Tragedia Familiar', 'Sofia Coppola', 2005, 3, 3),
('Códigos del Crimen', 'Martin Scorsese', 2019, 4, 1),
('La Oscuridad', 'James Wan', 2018, 1, 5),
('Aventuras del Norte', 'Ron Howard', 2011, 1, 6),
('Robo en el Espacio', 'Christopher Nolan', 2014, 1, 4),
('La Risa Prohibida', 'Adam McKay', 2022, 1, 2),
('Sombras del Amor', 'Greta Gerwig', 2023, 1, 3);
```

GO

-- Relaciones Cliente - Categoria (ejemplo de preferencias)

-- Nota: columnas en la tabla intermedia son (CategoriasId, ClientesId)

```
INSERT INTO ClienteCategoria (CategoriasId, ClientesId) VALUES
(1, 2), (2, 2),      -- Laura: Acción, Comedia
(3, 3), (4, 3),      -- Carlos: Drama, Ciencia Ficción
(5, 4), (6, 4),      -- María: Terror, Aventura
(1, 5), (4, 5),      -- Javier: Acción, Ciencia Ficción
(2, 6), (3, 6);      -- Lucía: Comedia, Drama
```

GO

2. Desarrollo de la Interfaz

Vamos a partir de esta interfaz básica que consta de una cabecera con filtros y un ListBox para mostrar los resultados:

```
<Window x:Class="Video_Club.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:local="clr-namespace:Video_Club"
    mc:Ignorable="d"
    Title="Gestión de Películas" Height="600" Width="900"
    WindowStartupLocation="CenterScreen" ResizeMode="NoResize">
    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>

        <!-- FILA SUPERIOR (Filtros y botones) -->
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10" Grid.Row="0">

            <!-- Combobox de Clientes -->
            <Label Content="Cliente: " FontWeight="Bold" Margin="10,10,5,10"/>
            <ComboBox Name="cmbClientes" Width="120" Margin="0,10,5,10"
                VerticalContentAlignment="Center"
                SelectedIndex="0"/>

            <!-- Buscador -->
            <Label Content="Título: " FontWeight="Bold" Margin="10,10,5,10"/>
            <TextBox Name="txtBuscarTitulo" Width="220" Margin="0,10,5,10"
                VerticalContentAlignment="Center"/>

            <!-- ComboBox de Categorías -->
            <Label Content="Categoría: " FontWeight="Bold" Margin="10,10,5,10"/>
            <ComboBox Name="cmbCategorias" Width="120" Margin="0,10,5,10"
                VerticalContentAlignment="Center"
                SelectedIndex="-1"/>
            <Button Name="btnReset" Content="R" Width="20" Height="20"
                Margin="5,10,10,10"/>

            <!-- Botón de agregar -->
            <Button Name="btnAgregar" Width="100" Height="30" Content="Agregar
Película" Margin="20,10,10,10"/>
        </StackPanel>

        <!-- LISTADO DE PELÍCULAS -->
        <StackPanel Grid.Row="1" Grid.Column="0">
            <!-- Cabecera manual -->
            <Grid Background="#EEE" Margin="0,0,0,5">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>
            </Grid>
        </StackPanel>
    </Grid>
</Window>
```

```

        </ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <TextBlock Text="Título" FontWeight="Bold" Grid.Column="0"
Margin="5,0" />
    <TextBlock Text="Cliente" FontWeight="Bold" Grid.Column="1"
Margin="5,0" />
    <TextBlock Text="Categoria" FontWeight="Bold" Grid.Column="2"
Margin="5,0" />
</Grid>

<ListBox Name="lstPeliculas" HorizontalContentAlignment="Stretch">
    <ListBox.ItemTemplate>
        <DataTemplate>
            <Grid Margin="2">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>

                <TextBlock Text="{Binding Titulo}" Grid.Column="0"
Margin="5,0" />
                <TextBlock Text="{Binding Cliente.Usuario}"
Grid.Column="1" Margin="5,0" />
                <TextBlock Text="{Binding Categoria.Clasificacion}"
Grid.Column="2" Margin="5,0" />
            </Grid>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>
</StackPanel>
</Grid>
</Window>

```

Ahora que tenemos la interfaz base vamos a darle logica a los filtros para poder filtrar peliculas.

Lo primero de todo es cargar los datos de la base de datos en la interfaz:

MainWindow.xaml.cs

```

using Microsoft.EntityFrameworkCore;
using System.Collections.ObjectModel;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;

```

```

using System.Windows.Navigation;
using System.Windows.Shapes;

namespace Video_Club
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        private List<Pelicula> todasPeliculas = new(); //
        Lista de peliculas
        private List<Cliente> todosClientes = new(); //
        Lista de clientes
        private List<Categoria> todasCategorias = new(); //
        Lista de categorias
        private ObservableCollection<Pelicula> peliculasVisibles = new(); // Lo
        que mostramos en la UI
        private ObservableCollection<Cliente> clientesVisibles = new(); // Lo
        que mostramos en la UI
        private ObservableCollection<Categoria> categoriasVisibles = new(); // Lo
        que mostramos en la UI

        public MainWindow()
        {
            InitializeComponent();
            // Asignamos los ItemSource a cada elemento
            cmbClientes.ItemsSource = clientesVisibles;
            lstPeliculas.ItemsSource = peliculasVisibles;
            cmbCategorias.ItemsSource = categoriasVisibles;

            // Cargamos los datos en la interfaz
            _ = CargarFiltroClientes();
            _ = CargarFiltroCategorias();
            _ = CargarPeliculas();
        }

        private async Task CargarFiltroClientes()
        {
            // Conectamos con la base de datos
            ServiceCliente service = new ServiceCliente();
            todosClientes = await service.Listar();

            // Cargamos los clientes en la observable collection
            clientesVisibles.Clear();
            foreach (var c in todosClientes)
            {
                clientesVisibles.Add(c);
            }
        }

        private async Task CargarFiltroCategorias()

```

```

{
    // Conectamos con la base de datos
    ServiceCategoria service = new ServiceCategoria();
    todasCategorias = await service.Listar();

    // Cargamos las categorias en la observable collection
    categoriasVisibles.Clear();
    foreach (var cat in todasCategorias)
    {
        categoriasVisibles.Add(cat);
    }
}

private async Task CargarPelículas()
{
    // Conectamos con la base de datos
    ServicePelicula service = new ServicePelicula();
    todasPelículas = await service.Listar();

    // Cargamos las películas en la observable collection
    películasVisibles.Clear();
    foreach (var p in todasPelículas)
    {
        películasVisibles.Add(p);
    }
    AplicarFiltros();
}

private void Filtros_Changed(object sender, RoutedEventArgs e)
{
    AplicarFiltros();
}

private void AplicarFiltros()
{
    // Extraemos la información de los filtros
    var filtroCliente = cmbClientes.SelectedItem as Cliente;
    string filtroTitulo = txtBuscarTitulo.Text.Trim().ToLower();
    var filtroCategoria = cmbCategorias.SelectedItem as Categoria;

    // Filtramos la colección con los datos puestos en el buscador
    var filtradas = todasPelículas.Where(p =>
        (filtroCliente == null || p.ClienteId == filtroCliente.Id) &&
        (string.IsNullOrEmpty(filtroTitulo) ||
p.Titulo.ToLower().Contains(filtroTitulo)) &&
        (filtroCategoria == null || p.CategoriaId == filtroCategoria.Id));

    // Recargamos la ObservableCollection con los datos que queremos mostrar
    películasVisibles.Clear();
    foreach (var p in filtradas)

```

```

    {
        peliculasVisibles.Add(p);
    }
}
}
}
}

```

Una vez hemos cargado los datos en la interfaz, tenemos que indicar a los combobox cómo queremos que se muestren, para ello hacemos los siguientes cambios:

MainWindow.xaml

```

<!-- Combobox de Clientes -->
<Label Content="Cliente: " FontWeight="Bold" Margin="10,10,5,10"/>
<ComboBox Name="cmbClientes" Width="120" Margin="0,10,5,10"
    VerticalContentAlignment="Center"
    SelectedIndex="0"
    DisplayMemberPath="Usuario"
    SelectedValuePath="Id"
    SelectionChanged="Filtros_Changed"/>

<!-- Buscador -->
<Label Content="Título: " FontWeight="Bold" Margin="10,10,5,10"/>
<TextBox Name="txtBuscarTitulo" Width="220" Margin="0,10,5,10"
    VerticalContentAlignment="Center"
    TextChanged="Filtros_Changed"/>

<!-- ComboBox de Categorías -->
<Label Content="Categoría: " FontWeight="Bold" Margin="10,10,5,10"/>
<ComboBox Name="cmbCategorias" Width="120" Margin="0,10,5,10"
    VerticalContentAlignment="Center"
    SelectedIndex="-1"
    DisplayMemberPath="Clasificacion"
    SelectedValuePath="Id"
    SelectionChanged="Filtros_Changed"/>
<Button Name="btnReset" Content="R" Width="20" Height="20"
Margin="5,10,10,10" Click="Reset_Click"/>

<!-- Botón de agregar -->
<Button Name="btnAgregar" Width="100" Height="30" Content="Agregar
Película" Margin="20,10,10,10"
    Click="btnAgregar_Click"/>
</StackPanel>

```

2.1 Edición de las películas

Para editar los datos de las películas vamos a implementar la funcionalidad de doble click dentro del listbox y llamar a una función que genere una nueva ventana con esos datos:

Editamos MainWindow.xaml

```

<ListBox Name="lstPelículas" HorizontalContentAlignment="Stretch"
MouseDownClick="lstPelículas_MouseDoubleClick">
    <ListBox.ItemTemplate>
        <DataTemplate>
            <Grid Margin="2">
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="*" />
                </Grid.ColumnDefinitions>

                <TextBlock Text="{Binding Titulo}" Grid.Column="0" Margin="5,0" />
                <TextBlock Text="{Binding Cliente.Usuario}" Grid.Column="1"
Margin="5,0" />
                <TextBlock Text="{Binding Categoria.Clasificacion}" Grid.Column="2"
Margin="5,0" />
            </Grid>
        </DataTemplate>
    </ListBox.ItemTemplate>
</ListBox>

```

Ahora añadimos al codebehind la llamada a la nueva ventana y le pasamos el Id de la película seleccionada para poder editar sus parámetros:

MainWindow.xaml.cs

```

private async void lstPelículas_MouseDoubleClick(object sender,
System.Windows.Input.MouseButtonEventArgs e)
{
    if (lstPelículas.SelectedItem is Película p)
    {
        var ventanaEditar = new EditarPelículaWindow(p.Id);
        bool? resultado = ventanaEditar.ShowDialog();

        if (resultado == true)
        {
            await CargarPelículas();
            AplicarFiltros();
        }
    }
}

```

2.1.1 Ventana editar

Creamos una ventana en la que mostrar los datos del objeto seleccionado y poder editarlo:

EditarPelículaWindow.xaml

```

<Window x:Class="Video_Club.EditarPelículaWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"

```

```

xmlns:local="clr-namespace:Video_Club"
mc:Ignorable="d"
Title="Editar Película" Height="340" Width="400"
WindowStartupLocation="CenterScreen" ResizeMode="NoResize">
<Grid Margin="10">
    <GroupBox Header="Datos de la Pelicula" Margin="0,10,0,0">
        <Grid Margin="10">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="150"/>
                <ColumnDefinition Width="*/>
            </Grid.ColumnDefinitions>

            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
            </Grid.RowDefinitions>

            <Label Content="ID:" Grid.Row="0" Grid.Column="0"
VerticalAlignment="Center"/>
            <TextBox Name="txtId" Grid.Row="0" Grid.Column="1" Margin="5"
IsReadOnly="True"/>

            <Label Content="Título:" Grid.Row="1" Grid.Column="0"
VerticalAlignment="Center"/>
            <TextBox Name="txtTitulo" Grid.Row="1" Grid.Column="1" Margin="5"/>

            <Label Content="Director:" Grid.Row="2" Grid.Column="0"
VerticalAlignment="Center"/>
            <TextBox Name="txtDirector" Grid.Row="2" Grid.Column="1" Margin="5"/>

            <Label Content="Año:" Grid.Row="3" Grid.Column="0"
VerticalAlignment="Center"/>
            <TextBox Name="txtYear" Grid.Row="3" Grid.Column="1" Margin="5"/>

            <Label Content="Categoría:" Grid.Row="4" Grid.Column="0"
VerticalAlignment="Center"/>
            <ComboBox Name="cmbCategoria" Grid.Row="4" Grid.Column="1" Margin="5"
                SelectedIndex="-1"
                DisplayMemberPath="Clasificacion"
                SelectedValuePath="Id"/>

            <Label Content="Cliente:" Grid.Row="5" Grid.Column="0"
VerticalAlignment="Center"/>
            <ComboBox Name="cmbCliente" Grid.Row="5" Grid.Column="1" Margin="5"
                SelectedIndex="0"
                DisplayMemberPath="Usuario"
                SelectedValuePath="Id"/>

```

```

        <!-- BOTONES -->
        <StackPanel Grid.Row="6" Grid.ColumnSpan="2" Orientation="Horizontal"
HorizontalAlignment="Center" Margin="0,20,0,0">
            <Button Name="btnGuardar" Content="Guardar Cambios" Width="150"
Height="30" Margin="10,0" Click="btnGuardar_Click"/>
            <Button Name="btnCancelar" Content="Cerrar" Width="100"
Height="30" Margin="10,0" Click="btnCancelar_Click"/>
        </StackPanel>
    </Grid>
</GroupBox>
</Grid>
</Window>

```

EditarPeliculaWindow.xaml.cs

```

using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Video_Club
{
    /// <summary>
    /// Lógica de interacción para EditarPeliculaWindow.xaml
    /// </summary>
    public partial class EditarPeliculaWindow : Window
    {
        public EditarPeliculaWindow(int peliculaId)
        {
            InitializeComponent();

            // Cargamos los datos de la pelicula si existe
            _ = CargarPelicula(peliculaId);

            // Cargamos los Combo Box
            _ = CargarClientes();
            _ = CargarCategorias();
        }

        private async Task CargarPelicula(int p_Id)

```

```

{
    // Conectamos con la base de datos
    ServicePelicula service = new ServicePelicula();
    var peli = await service.Listar(p_Id);

    // Cargamos las peliculas en la interfaz
    if(peli != null)
    {
        txtId.Text = peli.Id.ToString();
        txtTitulo.Text = peli.Titulo;
        txtDirector.Text = peli.Director;
        txtYear.Text = peli.Year.ToString();
        cmbCategoria.SelectedValue = peli.CategoriaId;
        cmbCliente.SelectedValue = peli.ClienteId; // si null -> 0 o nada
    }
    else
    {
        MessageBox.Show("Error al cargar los datos de la pelicula");
        this.Close();
    }
}

private async Task CargarClientes()
{
    // Iniciamos la lista de clientes
    List<Cliente> clientes = new List<Cliente>();

    // Conectamos a la base de datos
    ServiceCliente service = new ServiceCliente();
    clientes = await service.Listar();

    // Añadimos los clientes a la interfaz
    cmbCliente.ItemsSource = clientes;
}

private async Task CargarCategorias()
{
    // Iniciamos la lista de clientes
    List<Categoria> categorias = new List<Categoria>();

    // Conectamos a la base de datos
    ServiceCategoria service = new ServiceCategoria();
    categorias = await service.Listar();

    // Añadimos los clientes a la interfaz
    cmbCategoria.ItemsSource = categorias;
}

private async void btnGuardar_Click(object sender, RoutedEventArgs e)
{
    // Haríamos las comprobaciones de que todo esta cumplimentado
    adecuadamente

```

```

        if (string.IsNullOrEmpty(txtTitulo.Text) ||
            string.IsNullOrEmpty(txtDirector.Text) ||
            string.IsNullOrEmpty(txtYear.Text) || cmbCategoria.SelectedIndex == -1 ||
            cmbCliente.SelectedIndex == -1)
        {
            MessageBox.Show("ERROR: No se puede actualizar la película. Por
favor, rellene todos los campos");
        }
        else
        {
            // Conectamos con la base de datos
            ServicePelicula service = new ServicePelicula();
            var peli = await service.Listar(int.Parse(txtId.Text));
            if (peli != null)
            {
                peli.Titulo = txtTitulo.Text.Trim();
                peli.Director = txtDirector.Text.Trim();
                peli.Year = int.Parse(txtYear.Text);
                peli.CategoriaId = cmbCategoria.SelectedIndex + 1;
                peli.ClienteId = cmbCliente.SelectedIndex + 1;

                // Actualizamos la película
                var actualizada = await service.Actualizar(peli);
                if (actualizada)
                {
                    MessageBox.Show("Película actualizada correctamente.");
                    this.DialogResult = true;
                    this.Close();
                }
                else
                {
                    MessageBox.Show("Ha ocurrido un error en el proceso de
actualización.");
                }
            }
            else
            {
                MessageBox.Show("Ha ocurrido un error.");
                this.Close();
            }
        }
    }

    private void btnCancelar_Click(object sender, RoutedEventArgs e)
    {
        this.Close();
    }
}
}

```

Esta ventana esta configurada para que cuando actualice correctamente una película, devuelva un `True` al cerrarse y así podamos actualizar los datos de la interfaz principal a su vuelta.

2.1.2 Crear nueva película

Ahora vamos a dar funcionalidad al botón de nueva película para añadir una nueva a nuestra colección, para ello modificamos lo siguiente:

MainWindow.xaml

```
<!-- Botón de agregar -->
<Button Name="btnAgregar" Width="100" Height="30" Content="Agregar Película"
Margin="20,10,10,10"
Click="btnAgregar_Click"/>
```

MainWindow.xaml.cs

```
private async void btnAgregar_Click(object sender, RoutedEventArgs e)
{
    var ventanaCrear = new CrearPeliculaWindow();
    bool? resultado = ventanaCrear.ShowDialog();

    if (resultado == true)
    {
        await CargarPeliculas();
        AplicarFiltros(); // Reaplica los filtros activos
    }
}
```

Creamos una nueva ventana de WPF para meter el formulario de generar nueva pelicula:

CrearPeliculaWindow.xaml

```
<Window x:Class="Video_Club.CrearPeliculaWindow"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:Video_Club"
mc:Ignorable="d"
Title="Crear Película" Height="320" Width="400"
WindowStartupLocation="CenterScreen" ResizeMode="NoResize">
<Grid Margin="10">
<GroupBox Header="Datos de la Nueva Pelicula" Margin="0,10,0,0">
<Grid Margin="10">
<Grid.ColumnDefinitions>
<ColumnDefinition Width="150"/>
<ColumnDefinition Width="*/>
</Grid.ColumnDefinitions>

<Grid.RowDefinitions>
<RowDefinition Height="Auto"/>
```

```

        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

    <Label Content="Título:" Grid.Row="0" Grid.Column="0"
VerticalAlignment="Center"/>
    <TextBox Name="txtTitulo" Grid.Row="0" Grid.Column="1" Margin="5"/>

    <Label Content="Director:" Grid.Row="1" Grid.Column="0"
VerticalAlignment="Center"/>
    <TextBox Name="txtDirector" Grid.Row="1" Grid.Column="1" Margin="5"/>

    <Label Content="Año:" Grid.Row="2" Grid.Column="0"
VerticalAlignment="Center"/>
    <TextBox Name="txtYear" Grid.Row="2" Grid.Column="1" Margin="5"/>

    <Label Content="Categoria:" Grid.Row="3" Grid.Column="0"
VerticalAlignment="Center"/>
    <ComboBox Name="cmbCategoria" Grid.Row="3" Grid.Column="1" Margin="5"
        SelectedIndex="0"
        DisplayMemberPath="Clasificacion"
        SelectedValuePath="Id"/>

    <Label Content="Cliente:" Grid.Row="4" Grid.Column="0"
VerticalAlignment="Center"/>
    <ComboBox Name="cmbCliente" Grid.Row="4" Grid.Column="1" Margin="5"
        SelectedIndex="0"
        DisplayMemberPath="Usuario"
        SelectedValuePath="Id"/>

    <!-- BOTONES -->
    <StackPanel Grid.Row="5" Grid.ColumnSpan="2" Orientation="Horizontal"
HorizontalAlignment="Center" Margin="0,20,0,0">
        <Button Name="btnGuardar" Content="Añadir Pelicula" Width="150"
Height="30" Margin="10,0" Click="btnGuardar_Click"/>
        <Button Name="btnCancelar" Content="Cerrar" Width="100"
Height="30" Margin="10,0" Click="btnCancelar_Click"/>
    </StackPanel>
</Grid>
</GroupBox>
</Grid>
</Window>

```

CrearPeliculaWindow.xaml.cs

```

using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;

```

```

using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Shapes;

namespace Video_Club
{
    /// <summary>
    /// Lógica de interacción para CrearPeliculaWindow.xaml
    /// </summary>
    public partial class CrearPeliculaWindow : Window
    {
        private List<Cliente> todosClientes = new(); //
        Lista de clientes
        private List<Categoria> todasCategorias = new(); //
        Lista de categorias
        public CrearPeliculaWindow()
        {
            InitializeComponent();

            // Cargamos los Combo Box
            _ = CargarClientes();
            _ = CargarCategorias();
        }

        private async Task CargarClientes()
        {
            // Conectamos a la base de datos
            ServiceCliente service = new ServiceCliente();
            todosClientes = await service.Listar();

            // Asignamos los ItemSource a cada elemento
            cmbCliente.ItemsSource = todosClientes;
        }

        private async Task CargarCategorias()
        {
            // Conectamos a la base de datos
            ServiceCategoria service = new ServiceCategoria();
            todasCategorias = await service.Listar();

            // Asignamos los ItemSource a cada elemento
            cmbCategoria.ItemsSource = todasCategorias;
        }

        private async void btnGuardar_Click(object sender, RoutedEventArgs e)
        {

```

```

// Haríamos las comprobaciones de que todo esta cumplimentado
adecuadamente
    if (string.IsNullOrEmpty(txtTitulo.Text) ||
string.IsNullOrEmpty(txtDirector.Text) ||
string.IsNullOrEmpty(txtYear.Text) || cmbCategoria.SelectedIndex == -1 ||
cmbCliente.SelectedIndex == -1)
    {
        MessageBox.Show("ERROR: No se puede crear la película. Por favor,
rellene todos los campos");
    }
    else
    {
        // Conectamos con la base de datos
        ServicePelicula service = new ServicePelicula();
        Pelicula peli = new Pelicula()
        {
            Titulo = txtTitulo.Text.Trim(),
            Director = txtDirector.Text.Trim(),
            Year = int.Parse(txtYear.Text.Trim()),
            CategoriaId = cmbCategoria.SelectedIndex + 1,
            ClienteId = cmbCliente.SelectedIndex + 1
        };

        // Creamos la película
        var creada = await service.Insertar(peli);

        if (creada != null)
        {
            MessageBox.Show("Película creada correctamente!");
            this.DialogResult = true;
            this.Close();
        }
        else
        {
            MessageBox.Show("Ha ocurrido un error al crear la película.");
        }
    }
}

private void btnCancelar_Click(object sender, RoutedEventArgs e)
{
    this.Close();
}
}
}

```

El funcionamiento es similar al del formulario anterior de editar películas, pero en este caso las creamos.

2.1.3 Botón Reset

Este es un botón muy sencillo que resetea el filtro de categorías de la ventana principal, básicamente lo que hace es des-seleccionar la opción que esté y volver a aplicar los filtros:

MainWindow.xaml

```
<Button Name="btnReset" Content="R" Width="20" Height="20" Margin="5,10,10,10"
Click="Reset_Click"/>
```

MainWindow.xaml.cs

```
private void Reset_Click(object sender, RoutedEventArgs e)
{
    cmbCategorias.SelectedIndex = -1;
    AplicarFiltros();
}
```